Tensix NEO

High-level Architecture Specification

Revision 1.1

April 27, 2026

tenstorrent

Table of Contents

Introduction

The purpose of this document is to provide a comprehensive specification for the Tensix NEO high-level architecture (HAS). This document details the architectural design, functional blocks, interfaces, and characteristics of the Tensix NEO, serving as a foundational reference for hardware and software development teams. It aims to ensure a clear understanding of the design principles and implementation specifics necessary for the successful implementation and integration of the Tensix NEO into various systems and sub-systems.

Key components described in this micro architecture specification include:

- Tensix NEO cluster: The Tensix NEO cluster is a high-performance processing unit designed to deliver scalable and efficient computation. Each cluster integrates several key components:
- the compute core for executing complex operations
- a L1 memory for rapid local data access
- a data movement cluster to optimize inter-core communication and data transfer
- the network-on-chip (NoC) core for managing connectivity within and across clusters

Together, these elements enable the NEO cluster to achieve high throughput, low latency, and robust performance in demanding workloads.

In essence, the Tensix NEO HAS provides a blueprint for the hardware implementation, enabling hardware designers to build the chip and software developers to understand its performance characteristics and optimize their code accordingly.

Audience

- Firmware and embedded software developers
- SoC and hardware integration engineers

Some knowledge and understanding of RISCV core are assumed.

Related Documents

Document Title	Date
The RISC-V Instruction Set Manual Volume II: Privileged Architecture	December 4, 2021
NEO micro-architectural specifications	April 30, 2026
NEO Overlay specification	April 30, 2026
NEO network-on-chip (NOC) specification	April 30, 2026

Revision History

Date	Revision	Description
September 24, 2025	Rev 1.0	Initial release
April 30, 2026	Rev 1.1	Added NoC and data-movement, clock domains, and figures.
May 19, 2026	Rev 1.2	Added debug features and IDMA details

Tensix NEO Processor

The Tensix NEO™ processor is a highly parallel architecture designed for accelerating machine learning workloads, featuring a cluster of 4 compute cores, an advanced network-on-chip (NoC) for data distribution, and a substantial on-chip L1 memory. This document provides a high-level architectural specification, detailing the core functional blocks, memory hierarchy, data movement fabric, and key architectural features. It is intended for hardware and software engineering teams involved in the design, verification, and programming of the NEO architecture.

The NEO architecture offers flexible configurations, with 3 primary configurations which vary in functional safety support and matrix engine throughput, all optimized for improved performance, power efficiency, and flexible data movement:

- Tensix NEO (Quasar-QSR)
- Tensix NEO Auto (Trinity)
- Tensix NEO Robotics (upcoming)

Note:

- Tensix NEO™ is the branding for the newer generation of Tensix architecture.
- Tensix NEO compute core - a single Tensix NEO compute core

hereafter Tensix NEO compute core will be referred to as just “compute core.”

- Tensix NEO cluster - a cluster of 4 Tensix NEO cores + L1 memory + data movement cluster + NoC hereafter Tensix NEO cluster will be referred to as just “NEO cluster.”
- Data movement cluster is also known as “overlay.”

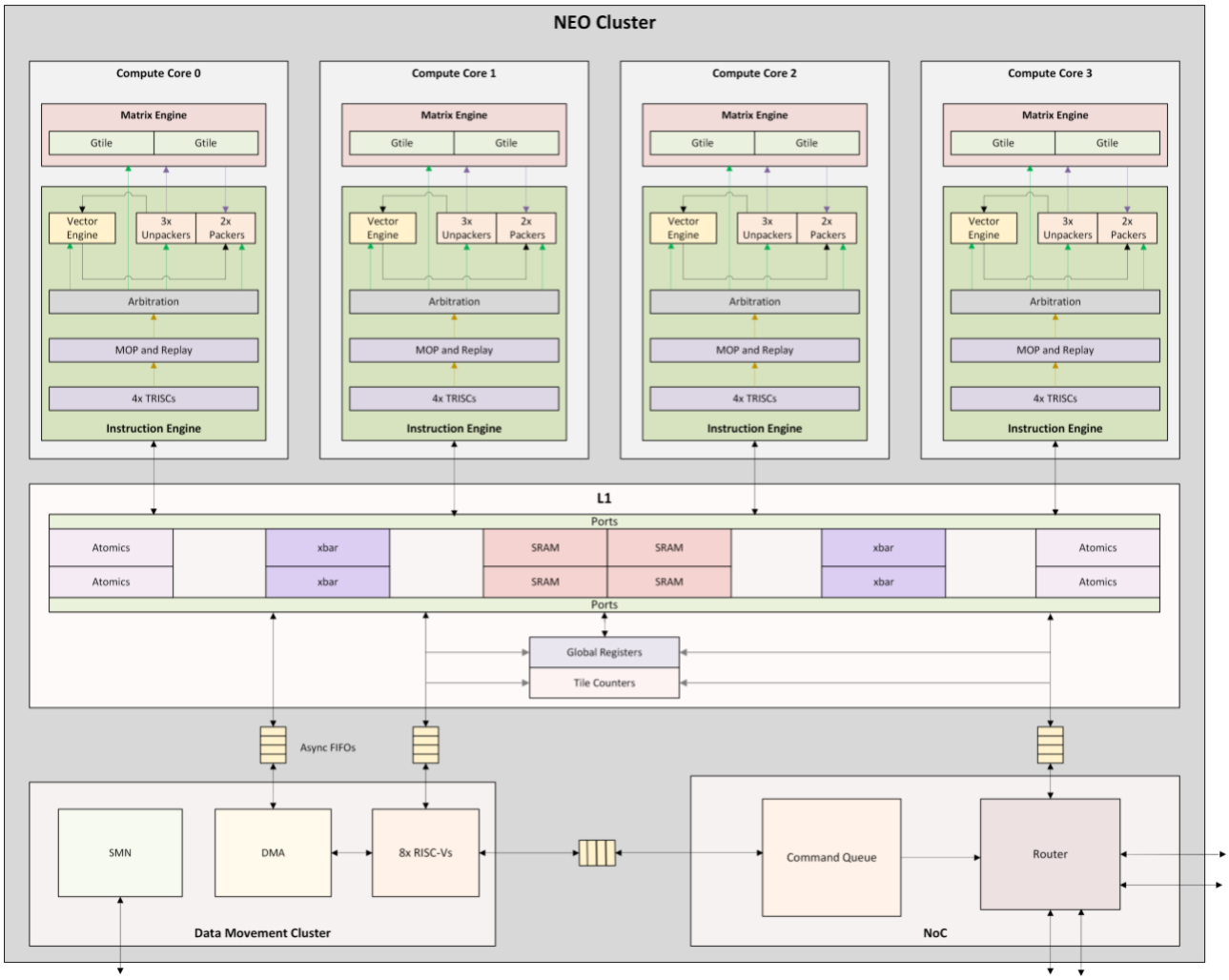

Figure 1: NEO Cluster Architecture. Trinity Configuration does not include SMN

Tensix NEO Cluster

The Tensix NEO cluster is a high-performance processing unit designed to deliver scalable and efficient computation. Each cluster integrates several key components: the compute core for executing matrix and vector operations, L1 memory for rapid local data access, a data movement cluster to optimize inter-core communication and data transfer, and the network-on-chip (NoC) for managing connectivity within and across clusters.

Compute Core

The compute core, see Figure 2, is a specialized processing unit designed to efficiently handle complex computational workloads. It integrates several key components, including a matrix engine and a vector engine for high-performance mathematical operations, as well as unpackers and packers for data formatting and movement. The core also features an arbitration mechanism that manages access to shared resources, four RISC-V (TRISC) cores for flexible control and programmability, and an instruction engine responsible for decoding and executing instructions.

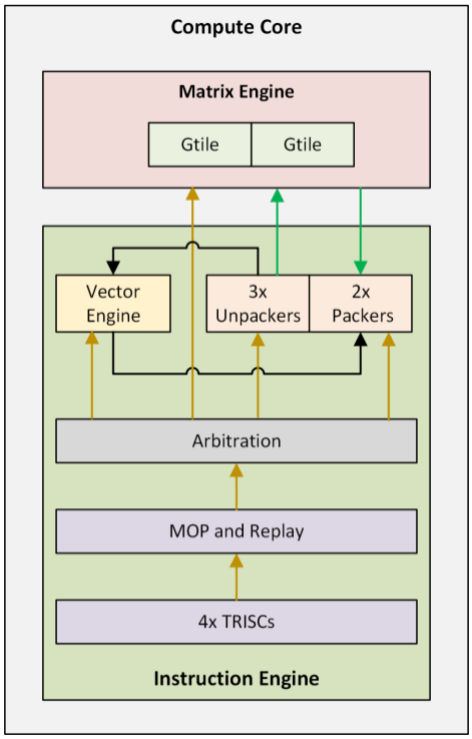

Figure 2: Compute core

Matrix Engine (FPU)

The Matrix Engine reads data from the source (src) register files and executes matrix-multiplication, elementwise add/subtract/multiply and max pooling instructions. For instructions that need accumulation, the destination register file (dest) is used to store accumulation data. Data can stay in dest for multiple accumulation cycles. The SrcA and Destination register file is present inside the matrix engine while the SrcB register file is instantiated in the Instruction Engine partition.

The matrix engine supports the BF16, FP16, TF32 and INT8 as input formats in src register files. Quasar configuration also supports MXFP4 format in src register-file. The accumulation may be done in BF16,

Configuration	Src Formats	Matrix-multiplication Throughput (MACs/clk per compute core)
Neo	TF32 [single fidelity-phase] FP16 [single fidelity-phase] BF16 INT8	2,048
	MXFP4	4,096
Neo Auto	INT8	2,048
	TF32 [single fidelity-phase] FP16 [single fidelity-phase] BF16	512

Table 1: Matrix engine throughputs

FP16, FP32 or INT32 precision.

The matrix engine in NEO outputs 8x16 datums per clock, where each datum is the result of a 16-way dot product for formats excluding MXFP4. For MXFP4, the dot-product is 32-way. For Neo Auto, the matrix engine outputs 4x16 datums per clock for integer formats with 32-way dot-products. For floating-point formats, it outputs 2x16 datums per clock with a 16-way dot-product each.

To optimize the area and throughput of the design, the matrix engine uses 8x8 multipliers in the data-path. This can support all formats with 8-bit or smaller mantissas in a single-cycle. For larger formats, the hardware allows the kernels to process the same data over multiple *fidelity-phases*. Each fidelity phase selects higher or lower bits from src register files, multiplies them, and accumulates them over dest register file. For TF32 and FP16 operands, a maximum of four fidelity phases may be needed to completely process all the input mantissa bits. The throughputs of the matrix engine for different formats and configurations is summarized in Table 1.

The SrcA and SrcB register files can store 32x32 datums of TF32 precision. Both are double-buffered to allow new data to be fetched into the register file while Matrix Engine is computing using the previous data. The result of the dot-product is accumulated over previous result stored in the destination register-file. Each output is rounded based on the accumulation format. The rounding modes supported are round-to-nearest with ties-to- even and stochastic rounding.

The destination register file has 1024 rows of 16 16-bit datums. For 32-bit accumulation, the destination register files uses two rows with effective size becoming 512 rows of 16 32-bit datums. Dest may also be used to provide operands and store results of the vector engine (SFPU). Similar to src register files, the dest is also double buffered, allowing matrix engine to produce new results, while the results from previous

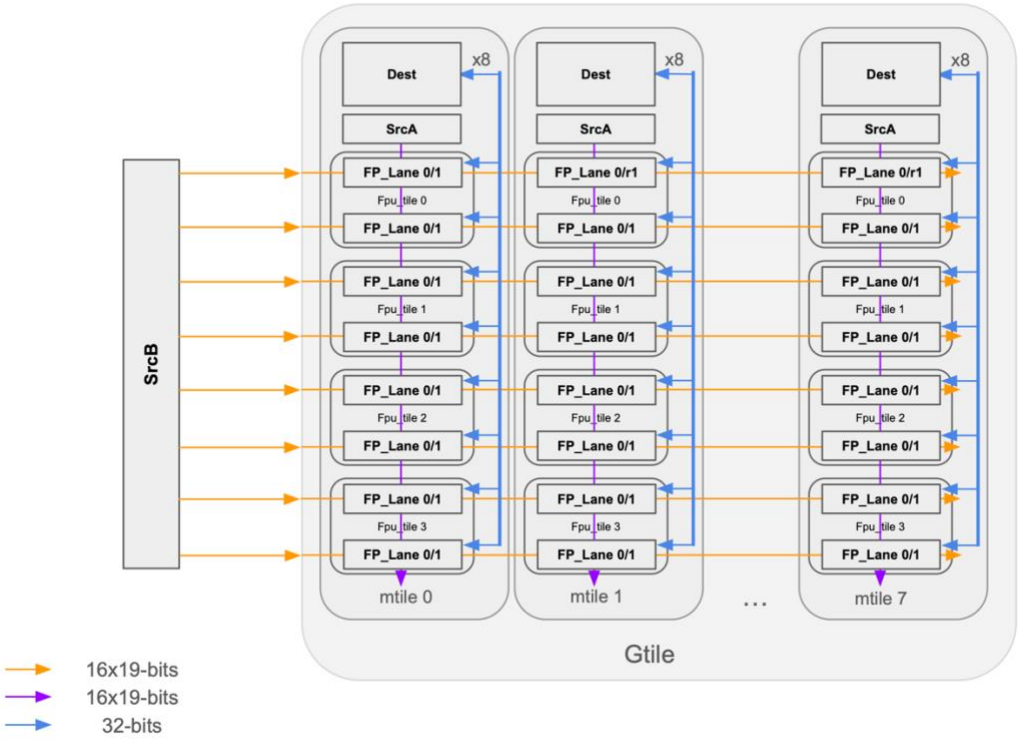

Figure 3: Gtile in Tensix Neo. Neo Auto has 2 FPU Tiles per Mtile.

accumulation are read out by the packer for packing into L1.

Both src and dest register file implement implied formats. This allows the register file to store the last format that was used to write into each of the double buffered banks. Any client that reads the register file can automatically use the data format that was used to write into the register file. This simplifies format programming in the kernels.

Gtile

GTile is a design partition created to improve physical design quality. It consists of a parameterized number of Mtile and dest instances. A single compute core has 16 mtile instances. These are partitioned into one or more gtiles. An example with two gtiles per compute-core is shown in Figure 2. The number of mtiles per gtile in such a case would be 8, as shown in Figure 3. A higher number of gtiles results in reduced logic per gtile. This can help increase turnaround time of physical design tools. Once a single Gtile has gone through the physical design stages, it can be instantiated multiple times.

Vector Engine

The vector engine (SFPU) implements a general-purpose ISA that can be used to execute vector operations. It can load and store data from the destination register-file or the SrcS register-file. Loaded data is stored in local registers (LREGs) within the SFPU. The SFPU supports FP32 precision multiply-add instructions and can be used to implement any transcendental operations using mathematical series expansion. NEO's Vector Engine also implements special instructions for single-cycle computation of reciprocal, exponential, square root, and hyperbolic tangent operations. The SFPU additionally supports INT32 arithmetic including addition/subtraction, logical operations, bit shifts, multiplication, and type conversions to and from floating-point. The SFPU also supports conditional execution where some of the lanes in the vector may not execute an instruction. This is achieved by setting or resetting a condition-code register per vector lane. The vector engine consists of 32 SFPU lanes. The 32 SFPU lanes are organized into a 4x8 grid (shown in Figure 4). Since both the destination and SrcS register files are 16 datums wide, any load from Dest/SrcS will fetch odd or even columns depending on the LSB of the load address. Stores work similarly by only updating even or odd columns in Dest/SrcS.

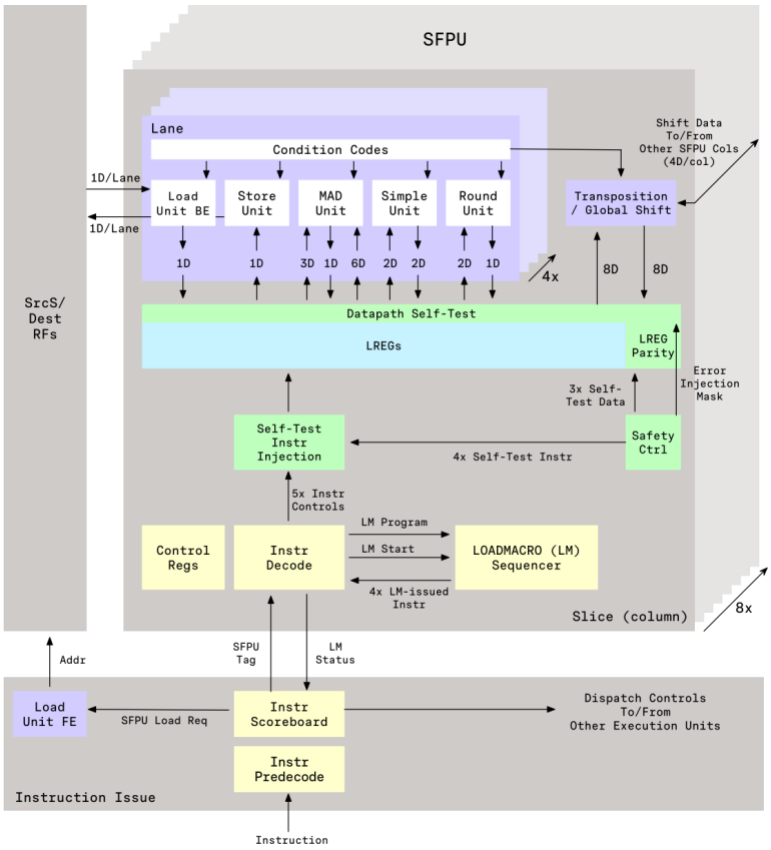

Figure 4: SFPU execution unit

Instruction Engine

A single Tensix core is composed of an instruction engine and one or more gtiles. The instruction engine, see Figure 5, includes 4 TRISC cores, TDMA block, config and sync execution units and instruction arbitration. Each TRISC core is a single issue in-order RiscV core. Mop and replay buffers are part of the instruction issue logic and are used to allow multiple preprogrammed instructions and/or loops to be executed using a single TRISC instruction. This substantially reduces the instruction delivery bottlenecks and allows small single issue Risc cores to keep the Tensix execution engines busy.

The instruction engine also includes the register files (SrcB, SrcS and Metadata) and the associated Vector Engine. The TDMA block, depicted in TDMA section, includes the unpackers, packers and the THCON execution engine. The *dma_srca* and *dma_srcb* blocks perform data-format conversions on-the-fly as new data is being written into register files. The *dest_adapter* is used to select even or odd columns based on the address indexed for the SFPU load/store data. This will be described in more detail in the Vector Engine section.

Tensix RISC (TRISC)

Each compute core has 4 single issue TRISCs that are used for executing low level kernels (LLKs). These LLKs are written in C with additional intrinsic TTI instructions. The TTI instructions pass through the TRISC pipeline and are written into the instruction buffers in the Tensix instruction engine. The TRISCs support

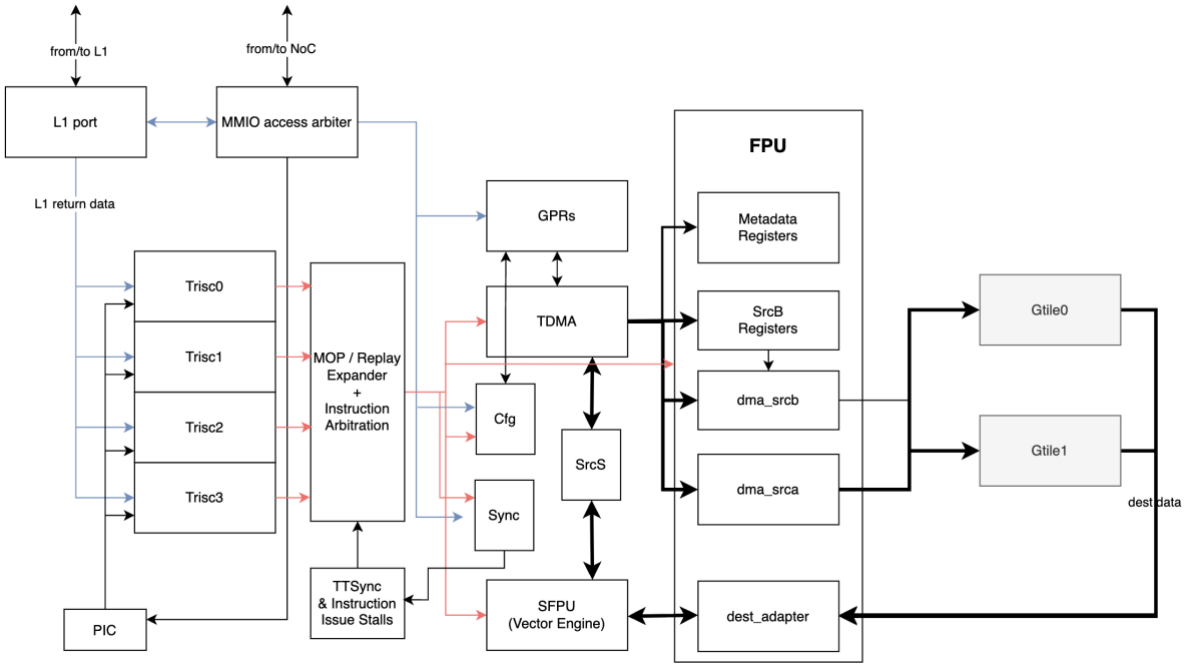

Figure 5: Instruction engine. Shaded blocks represent neighboring PD partitions.

IFAMO RiscV extensions. One of the TRISC cores, Trisc0, also supports the vector extension with a 128-bit vector unit.

The TRISC cores have MMIO access to GPRs, configuration and destination registers. Each TRISC core implements configurable data and instruction prefetching. A memory management unit within TRISC cores can be configured to define which address ranges are cacheable in the data cache.

The TRISC cores also support interrupts through set of maskable interrupts raised by a programmable interrupt controller (PIC). Each compute core has several hardware and software interrupts. The hardware interrupts can be used to immediately enter an interrupt handler in cases of a timeout, or after every instruction is issued from the Tensix instruction buffer. Other hardware interrupts include every time a new tile is unpacked into *SrcA*, *SrcB* or *SrcS* registers.

Instruction Pipeline

TRISC issues 32-bit Tensix instructions in program order to the Tensix instruction pipeline. The Instruction Pipeline uses Macro-Operation Decoder (MOP Decoder) to determine if the instruction is a MOP instruction. MOP instructions allow multi-level for-loops pre-programmed in MOP configuration registers to be issued using a single instruction, significantly reducing the instruction fetch requirements for the TRISC cores.

After MOP stage, the original TTI instruction or the MOP-inserted TTI instruction is decoded to see if it is a Replay instruction. Replay instructions can issue multiple contiguous instructions, pre-programmed in Replay Buffers, to be issued using a single instruction. If a replay instruction is detected, a state machine starts issuing the required number of instructions from the replay buffer. The instructions are placed in per-thread instruction buffers.

Instructions from different threads are arbitrated over and issued to corresponding execution units at the rate of 1 instruction per clock. During instruction issue, hardware also detects for scoreboard and double-buffering (*DValid*) stalls. It also creates internal state needed for the downstream execution units.

The Tensix-TRISC Sync (*TTSync*) detects and prevents data hazards between Tensix instructions and other memory-mapped accesses by the TRISC (in contrast, the Instruction Issue deals with data hazards between Tensix instructions only).

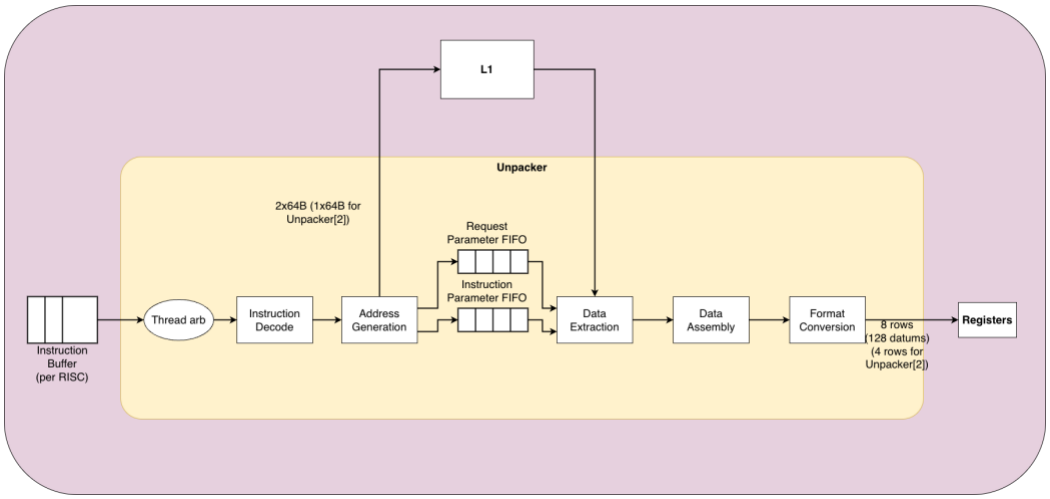

Figure 6: Unpacker execution pipeline

Unpackers

The unpacker is an execution engine that is present inside the TDMA block. It is responsible for fetching data from the L1 and bringing it into the register files for compute. The unpacker can also enable format conversion while bringing in data. NEO supports multiple unpack instructions that are implemented by the unpacker. These instructions can be used to fetch a tile (32x32 datums), a face (16x16 datums) or a row (1x16) datums into the register file. Strided instructions are also supported that allow fetching from non-contiguous rows in the L1. Lastly, since the matrix engine prefers a tilized data-layout, a tilize instruction in the unpacker can convert row-major data to 32x32 tiles in the src register files. A block diagram of the unpacker is shown in Figure 6.

There are three unpackers in the NEO architecture, namely unpacker0, unpacker1 and unpacker2. Unpacker0 is responsible for fetching data into the *SrcA* and the Destination register file in the matrix engine. The Unpacker1 is responsible for fetching data into the *SrcB* and the metadata register file in the Matrix Engine. The metadata register file is used by the matrix-multiplication instructions with structured sparsity. The Unpacker2 is used to fetch data into the *SrcS* register-file for the Vector Engine.

The unpacker determines the L1 fetch address based on the buffer descriptor registers programmed in the configuration block and the index specified in the instruction. The src register-file may be written at different locations depending on the index provided by the instruction. The **INC* versions of the instructions use a pair of counters per unpacker to determine the L1 fetch address and the src row address. The src register-file cannot update a partial row. The unpacker0 and 1 have an L1 bandwidth of 128B per clock while Unpack2 has a bandwidth of 64B per clock.

Table 2 summarizes the Unpack instructions supported by the Neo.

Instruction	Fetch Granularity (xdim x ydim x zdim)
UnpackRow	16x1 datums.
UnpackFace	16x16x1, 16x8x1,16x4x1, 16x2x1 and 16x1x1(non-MX only) datums
UnpackTile	16x16x4, 16x16x1, 16x8x1,16x4x1, 16x2x1 16x1x1 for non-MX Ability to limit fetch/store for unpack/pack to power-of-two xdims
UnpackTilize	16x16x4 datums
UnpackStrided	16x8 datums with the ability write 1 of the 8 rows, leaving other rows unchanged or overwriting other rows with a constant value. Each row can be at a uniform stride in the L1 or non-uniform strides programmed in configuration registers.
RVUnpack	16x16x4,16x16x1, 16x8x1, 16x4x1, 16x2x1, 16x1x1 (non-MX only) Can also support tilize. No configuration registers need to be programmed for this instruction.

Table 2: Summary of Unpack Instructions

Packers

The packer is an execution engine that is responsible for reading data from the destination and *SrcS* register files and storing to the L1. It is part of the TDMA block along with unpackers. A block diagram of packer is shown in Table 1. The packer functionality is a complete dual of unpacker. Packer can perform format conversions on the fly and can also round the data in case L1 format has lower precision than the register file format. The rounding may be round-to- nearest with ties-to-even or stochastic rounding. Packer can also perform additional operations on the fly, such as some ReLU flavours and edge masking. Edge masking is used to mask some of the rows and update the rows with value of 0 or negative infinity. Packer can also send stores to the L1 and request L1 accumulation. L1 accumulation uses the L1 floating-point atomics pipeline. There are two packers in NEO, one for the reading data from destination register file and the other for reading data from the *SrcS* register file. The two packers have a bandwidth of 128B and 64B

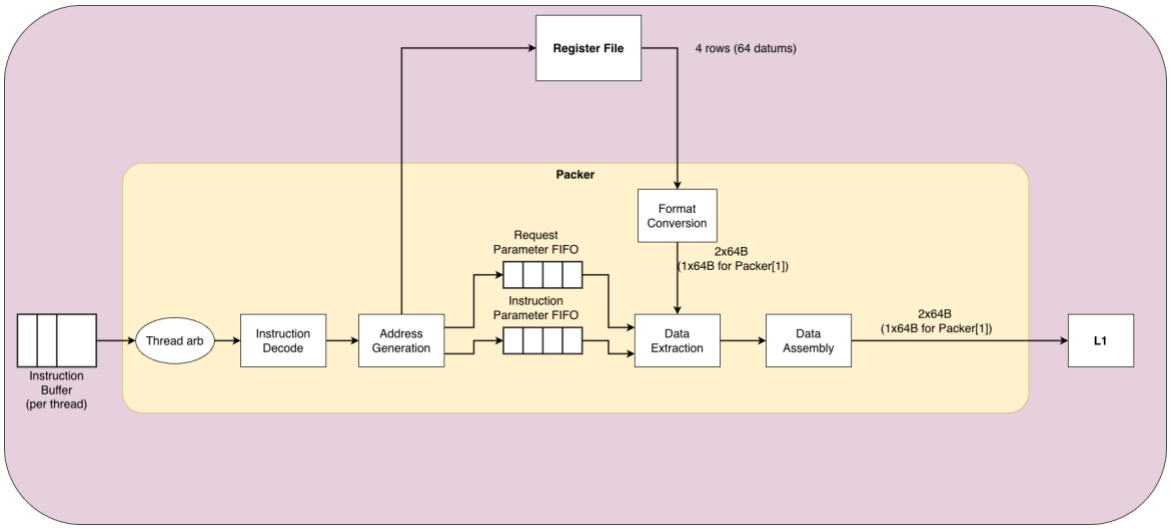

Figure 7: Packer execution pipeline

per clock, respectively. A summary of instructions supported by the packer is shown in Table 3.

Data-formats

The unpacker and packer can support multiple formats in the L1. These formats can be converted to a subset of formats supported by the register files. The unpacker formats and possible conversions are listed in Table 4. Packer formats supported by NEO are listed in Table 5.

Instruction	Store Granularity (xdim x ydim x zdim)
PackRow	16x1 datums
PackFace	16x16x1, 16x8x1,16x4x1, 16x2x1 and 16x1x1 (non-MX only) datums
PackTile	• 16x16x4, 16x16x1, 16x8x1,16x4x1, 16x2x1 • 16x1x1 for non-MX • Ability to limit fetch/store for unpack/pack to power-of-two xdims
PackUntilize	16x16x4 datums
PackStrided	16x4 datums with the ability write 1 of the 4 rows, leaving other rows unchanged or overwriting other rows with a constant value. Each row can be at a uniform stride in the L1 or non-uniform strides programmed in configuration registers. Only supported for non-MX formats
RVPack	16x16x4,16x16x1, 16x8x1, 16x4x1, 16x2x1, 16x1x1 (non-MX only) Can also support tilize. No configuration registers need to be programmed for this instruction.

Table 3: Summary of Pack instructions

Input Format (L1)	Output Format (register files)
Float32	Float32 (Unpack-to-Dest and Unpack-to-SrcS only) TF32 (unpacked into 32b Float32 containers in Dest and SrcS; unpacked into 19b TF32 containers in SrcA and SrcB), Float16, Float16_B
TF32	TF32 (unpacked into 32b Float32 containers in Dest and SrcS; unpacked into 19b TF32 containers in SrcA and SrcB), Float16, Float16_B
Float16, Float16_B FP8R, FP8P MXFP8R, MXFP8P MXFP6R, MXFP6P MXINT8, MXINT4 MXINT2	Float16, TF32 (not supported for Unpack-to-Dest and Unpack-to-SrcS), Float16_B
MXFP4	Float16, TF32 (not supported for Unpack-to-Dest and Unpack-to-SrcS), Float16_B, MXFP4_2x_A (not present in Neo Auto, not supported for Unpack-to-Dest and Unpack-to-SrcS), MXFP4_2x_B (not present in Neo Auto, not supported for Unpack-to-Dest and Unpack-to-SrcS)
INT32	INT32 (Unpack-to-Dest and Unpack-to-SrcS only)
INT8	INT8, INT8_2x (only present in Neo Auto, not supported for Unpack-to-Dest and Unpack-to-SrcS)
UINT8	UINT8 UINT8_2x (only present in Neo Auto, not supported for Unpack-to-Dest and Unpack-to-SrcS)
INT16	INT16 (also used for SFPU UINT16 input)
INT4	INT8, INT8_2x (only present in Neo Auto, not supported for Unpack-to-Dest and Unpack-to-SrcS)
UINT4	INT8, UINT8, INT8_2x (only present in Neo Auto, not supported for Unpack-to-Dest and Unpack-to-SrcS), UINT8_2x (only present in Neo Auto, not supported for Unpack-to-Dest and Unpack-to-SrcS)

Table 4: Supported L1 and Src formats for unpackers

Input Format (Dest/SrcS register file)	Output Format (L1)
UINT8	UINT8
INT32	INT32 INT8 UINT8
INT16 (also used for SFPU UINT16 output)	INT16
INT8	INT8
Float32	Float32 TF32 (round man to 10 bits; datum stored as 32-bits in L1 for alignment reasons) Float16, Float16_B FP8R, FP8P MXFP8R, MXFP8P, MXFP6R, MXFP6P, MXFP4 MXINT8, MXINT4, MXINT2
Float16, Float16_B	Float16, Float16_B FP8R, FP8P MXFP8R, MXFP8P, MXFP6R, MXFP6P MXFP4, MXINT8, MXINT4, MXINT2

Table 5: Supported Packer format conversions

Sync Unit

The Sync Unit supports instructions that may be used to synchronize different threads using mutexes and/or semaphores. The TRISC cores and the Tensix execution units typically operate in an asynchronous manner where the TRISCs can be further ahead in the kernel. This allows the TRISC cores to run-ahead and fetch more TTI instructions that can be issued to the Tensix execution units. In some situations, it is required to synchronize the TRISC with its corresponding instructions in the execution units. The Sync Unit also executes instructions that enable such synchronization.

Config Unit

The Config unit is responsible for managing configuration state and executing instructions that modify or query the configuration state of the tensix cores. The configuration state is used to modulate the behavior of different execution engines. Some of the features implemented using configuration state are input and

output data formats of the unpackers, matrix, vector and the packers. A part of the state called Buffer Descriptors is also used to define the properties of the different tensors allocated in the L1. These properties include L1 addresses, size of tensors and storage format. The Config Unit allows memory-mapped access to the configuration state for the TRISC cores, as well as TTI instructions that can read, write or read-modify-write configuration bits.

L1

The L1 is a highly configurable, multi-port, multi-bank local memory subsystem designed for low-latency, high-throughput simultaneous access across numerous independent clients. Organized as a hierarchy of super-banks, banks, and sub-banks at 16-byte access-size, the L1 provides a large array of independently addressable SRAM resources capable of servicing many concurrent accesses per cycle. Memory capacity, bank count, and port configuration are all parameterizable at design time. All data is optionally protected by SECDED ECC with single-bit error correction performed inline. The memory functions as a directly addressed local memory scratchpad rather than a hardware-managed cache.

L1 implements a partitioned crossbar between the ports and the banks that significantly reduces the routing complexity of the crossbar. However, the partitioned crossbar limits which banks may be accessible to each port on a given clock cycle. The crossbar is designed with FIFOs to store requests for multiple cycles and not back-pressure the clients immediately. The depth of these FIFOs is adjusted based on typical access patterns. The L1 also implements an address hashing function to interleave a sequence of addresses to different banks and sub-banks. In many cases, the address hashing function is critical to ensure a distribution of requests across multiple banks. The address hashing function can be updated using configuration registers. The L1 access patterns depend on both the kernels executing and the L1 interleaving function. In general, L1 FIFO depths, response buffer size and hashing functions are optimized for best power-performance-area (PPA) characteristics.

The L1 can issue requests to the sub-banks out-of-order with respect to the client request order. This flexibility mitigates the crossbar limitation of only being able to access a quarter of the banks per clock for a given client. A request may wait for 3 cycles, before it becomes eligible for issue to the sub-banks, if no older requests are outstanding. On a given cycle, a younger access request may be eligible for issue to the sub-banks, while older requests wait for their turn on the crossbar. The L1 ports maintain a response reordering buffer that is used to ensure that all responses are still sent to clients in client request order.

Some clients may execute a critical section of code that requires L1 updates to happen in-order with the client order. For such cases, the L1 implements CSRs that define the address ranges where ordering must be maintained. The ordering options are enumerated below:

1. ALL_IN_ORDER: No-reordering (sequentially consistent)

2. WRITE_ORDER: No-reordering of writes, reads can be reordered
3. RW_BARRIER_ORDER: reordering amongst reads and writes but reads not allowed to pass writes or vice versa.

The 16B width of sub-banks allows individual 16B requests to be scheduled on the 64B crossbar. While each sub-bank is 16B wide, the partitioned crossbar limits which requests can be issued to the 4 sub-banks in a bank.

Atomics

L1 also has atomics support to implement floating point accumulation, RiscV Atomics instructions as well as Thread controller (THCON) atomic instruction set. Any client can issue atomic operations to L1 (including floating point atomics).

Error-correction

Single-bit ECC errors are corrected on the fly by the L1 logic. A maskable interrupt may be raised to the Data-movement cores to allow them to perform a read-modify-write on the address to correct single-bit errors. Double-bit errors may also raise an interrupt. For *Neo Auto*, these errors are also reported over the Error detection and correction (EDC) bus.

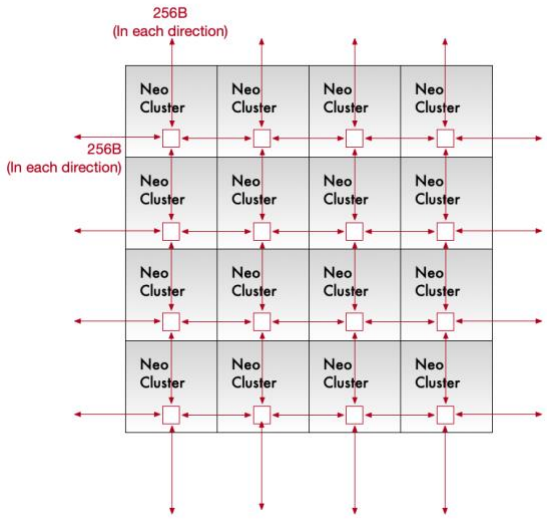

Figure 11: 4x4 Bidirectional Mesh Network Topology

NoC

NoC is based on the 2D bidirectional mesh topology. A mesh consists of a regular 2D array of endpoints, with adjacent nodes connected in each dimension. Consequently, routers have 5 input and output ports each (local endpoint interface, X-dimension ring, and Y-dimension ring).

The logical view of a 4x4 mesh network topology is shown in Figure 11, and the connectivity in Tensix tile is shown in Figure 12. In each node, NoC implements bidirectional connectivity in 3 directions: the mesh X and Y dimensions connecting to the adjacent tiles, and the local NIU (network interface unit) connecting to the local Tensix SRAM memory ports.

The NOC implements a Tenstorrent internal Quasar Noc Protocol (QNP). QNP implements a Wormhole flow-control mechanism. The NoC routers implement credit/debit based data interfaces between routers in each direction. The physical distance between the routers impacts the router design by increasing the number of credits the hardware must provide for to hide latency. Any repeater stages between NoC routers consequently increase the credit/debit related buffering needed in the NoC routers.

The NoC can be extended beyond a single-chip to route data over die-to-die interfaces, effectively creating a single larger system using multiple chiplets. Ideally, the final NoC grid implemented is rectangular. This avoids potential deadlock issues. The NoC also support software-specified routing (*dynamic routing*) where the routing path may be described explicitly by software. This allows the software to potentially avoid hot-spots.

While a grid of NEO clusters is expected to be rectangular, other components attached to the mesh may create a top-level non-rectangular grid. Non-rectangular grids can create deadlocks in dimension-order

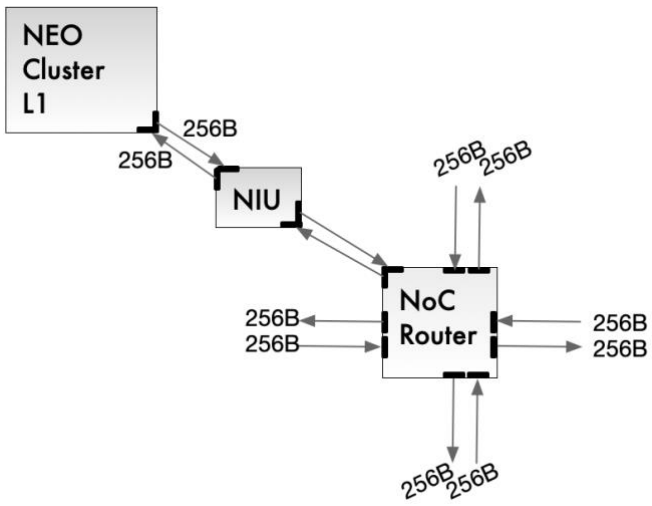

Figure 12: Mesh NoC Connectivity with Tensix SRAM in a Tensix Cluster Tile

routing protocol. The NEO NOC support *up-mesh-down* (UMD) routing for such special cases. UMD allows a sub-mesh of the overall mesh to be routed using dimension-order routing, while non-rectangular sub-meshes send traffic to the closest mesh node. Additionally, the NoC also supports flexible dimension-order routing per virtual channel where some virtual channels may do XY and some do YX routing. NoC supports AXI4 for endpoints for third-party IP.

NoC is a must-yield block in NEO cluster. The remaining NEO cluster partitions may be harvested out in case of defects. The NOC coordinates need to be re-programmed to remove the harvested cores. The NOC of the harvested core still needs to route the packets.

The NoC latency to each of its nearest neighbors is 10 cycles in the absence of any congestion. Virtual channels are supported to handle different classes of traffic and reduce congestion impact. There are a total of number of 16 virtual channels in NoC. Eight of these are dedicated for unicast traffic, four for multi-cast and four for responses. The NoC supports both posted and non-posted transactions to provide software flexibility. The non-posted transactions use the same NoC as data and hence can impact the performance of data transfers. Non-posted transactions are not supported with dynamic routing.

The NOC hardware supports a maximum packet size of 64KB. Transactions larger than this limit are automatically broken into smaller packers by the hardware. Finally, NOC transactions can by byte-aligned for non-atomic traffic. The maximum packet size may be configured to a smaller value (8KB, 16KB) by updating a configuration register. Smaller packet sizes can reduce NoC congestion effects for some traffic scenarios that involve head-of-line blocking. For atomics, the minimum granularity is 4B and they must be 4B aligned as well.

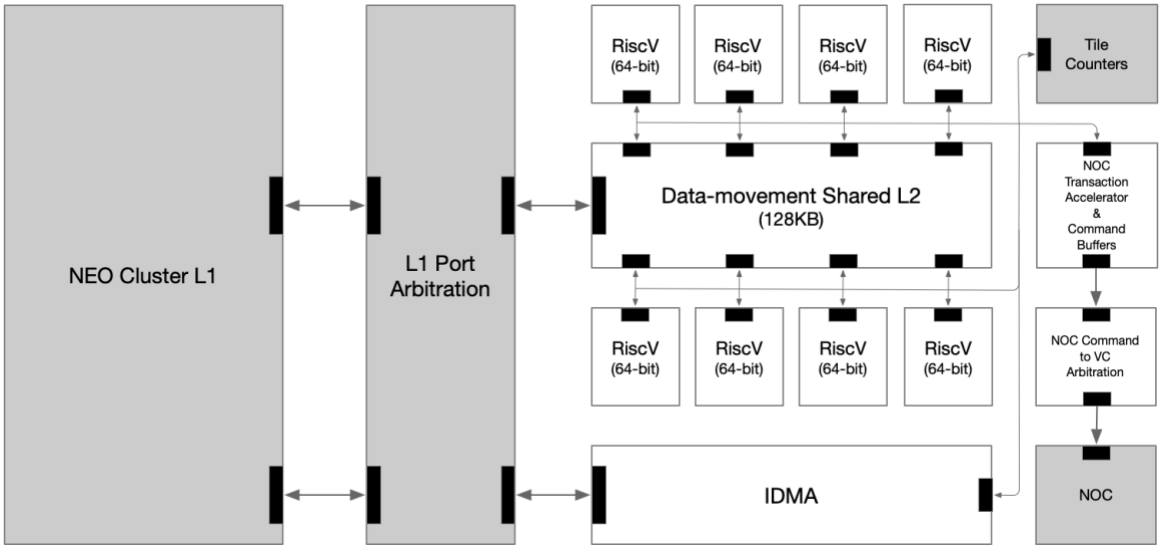

Figure 13: Overlay Top (shaded blocks are neighboring partitions)

Data Movement/Overlay

Eight RISC cores are instantiated along with data movement accelerators for each RISC. The RISCs are meant to be used as programmable DMA engines capable of issuing arbitrary data movement instructions. Tensix architecture enables asynchronous execution of data-movement operations with compute kernels. This allows overlap of compute and data-movement kernels without impacting performance. It also enables significant complexity in data-movement kernels which is extremely important for handling AI workloads.

Figure 13 shows a high-level block-diagram of the data-movement core complex. It consists of 64-bit RISC cores. These cores support integer operations including multiplication and division. Unlike TRISC cores, these cores do not support floating-point, vector or atomic instructions. The eight cores have private L1 caches and shared a unified L2 cache. The L2 cache is backed by NEO cluster L1.

The NOC Transaction Accelerator enables fast address-calculation without involving RiscV cores and also provides support for random access patterns pre-defined by software using the scatter-lists. Some typical scatter patterns can be generated on-the-fly by the accelerator as well. The IDMA engine enables data-movement within the NEO Cluster L1. It can do up to two 64B transactions per clock, where each transaction reads 64B from NEO cluster L1 and writes it back at a different location in the cluster L1.

The data-movement kernels executing on these cores can program the NoC command buffers to initiate transactions on the NoC. The command buffer is visible to the programmer as memory-mapped space and can be updated using RiscV store instructions. It can take multiple cycles to program the command buffer

for a single transaction. The software may do optimizations to re-use the same command buffer configuration to issue multiple transactions by using address counters or selectively updating the command buffer registers. Common NoC transactions can issue custom RiscV instructions to reduce the command buffer programming overhead. These custom instructions include uni-cast load and store instructions.

The data-movement cores also integrate IDMA engines that can be used to perform L1 to L1 data transfers within a single NEO cluster. The transfers can be triggered by the data movement cores and each instruction specifies source and destination addresses, length of transfer and stride patterns. In the Dispatch Engine, these IDMA engines are extended to allow format conversions during L1-L1 transfers. This is useful to allow dispatch-engines to fetch data from pinned memory on host and do fast format conversions to formats needed for compute and vice-versa. Since the host data is typically in float32 format, the format conversion logic is limited to conversion to/from float32 formats. MX formats are not natively supported on host, hence these conversions are not supported in the dispatch engine.

The prefetcher and dispatcher flow is expected to be as follows:

- Prefetcher can fetch data from host into L1
- Prefetcher kernel can issue an IDMA op for L1-L1 data movement with format conversion
- Dispatcher can read the converted data and send it to host or Neo clusters.

The IDMA engine uses unpacker and packer logic while reading and writing data from the L1, respectively. Format conversions can be performed by providing a combination of read and write formats through configuration registers for the unpacker and packer gaskets. A table for possible format conversions is highlighted in the Table 6.

IDMA Input/Output format	Unpacker gasket formats (in/out)	Packer gasket formats (in/out)
Float32/Float16B	Pass-through	Float32/Float16B
Float32/Float16A	Pass-through	Float32/Float16A
Float16A/FP8P	Pass-through	Float16A/FP8P
Float16A/FP8R	Pass-through	Float16A/FP8R
Float16B/FP8P	Pass-through	Float16B/FP8P
Float16B/FP8R	Pass-through	Float16B/FP8R
Float32/FP8P	Pass-through	Float32/FP8P
Float32/FP8R	Pass-through	Float32/FP8R
INT32/INT8	Pass-through	INT32/INT8 (no descaling)
INT32/UINT8	Pass-through	INT32/UINT8 (no descaling)
FP8P/Float16A	FP8P/Float16A	Pass-through
FP8R/Float16A	FP8R/Float16A	Pass-through
FP8P/Float16B	FP8P/Float16B	Pass-through
FP8R/Float16B	FP8R/Float16B	Pass-through
Float16/Float32	Float16/Float32	Pass-through
Float16B/Float32	Float16B/Float32	Pass-through
FP8P/Float32	FP8P/Float32	Pass-through
FP8R/Float32	FP8R/Float32	Pass-through
Others	Pass-through	Pass-through

Table 6: Format conversions available using IDMA in Dispatch Engine

Tile Counters

Tile-counters are used for synchronization between the Noc/data movement cores and the compute cores. Tile counters are incremented as NoC brings new tiles into the L1. The actual increment is done by the data movement kernels executing on data movement cores. There is a separate set of tile counters for every compute core. The unpacker trisc in each compute core can poll on (or use TTI_WAIT instruction) the tile-counter value to wait for new tiles to become available. When the tile-counter has a non-zero value, the unpacker can start fetching new data from the L1 into the source registers for compute. The tile-counter can be decremented by the unpacker TRISC using MMIO or TTI_POP instruction, when the kernel has consumed data.

Packer and data movement cores also synchronize using the tile counters. As packer produces new tiles in the L1, it increments a *tiles_available* counter. This can be done using MMIO writes or using TTI_PUSH instructions. Packer must also wait for buffer space to be available before writing new data into the L1. This can be done using the TTI_WAIT_FREE instruction. The data movement cores will receive an interrupt if tiles have been produced by the packer. The interrupt can be configured to be triggered when one or more tiles are available in the buffer. The interrupt-handler may program the NoC to send the data to another NoC coordinate.

All buffer management is expected to be done through the tile-counters. This includes intermediate buffers that do not need any NoC transactions. An example of such intermediate buffers are the outputs of intermediate operations in fused ops. The data movement cores can synchronize the output of one op to the input of the dependent op. For cases where the four compute cores are executing different ops of the same fused operation, the results from one Tensix may be needed by another Tensix. The tile-counter based synchronization will be used to synchronize all the compute cores.

Both the compute cores and the data movement cores need low-latency access to the Tile-counters. However, the physical distance between them, and the required clock-domain-crossing (CDC) fifos between them can significantly increase the round-trip request latency. This is alleviated by maintaining two copies of the tile counters, one in the AI clock domain in the L1 partition, and the other in the data movement cores partition. The hardware ensures coherency between the two copies of tile counters. There are a total of 32 tile-counters per compute core available in NEO. data movement cores can access the tile-counters of all compute cores.

Error Detection and Correction

Neo Configuration

Neo configuration supports ECC on the L1 SRAMs and the local data memory (LDM) instances of TRISC cores. The LDM supports byte-level ECC while the L1 banks support ECC at 16B granularity. ECC is only computed on the data bits in the TRISC cores. For L1, ECC is computed using both address and data-bits.

Instruction caches in the TRISC cores support parity protection. Single-bit errors and parity errors can raise a maskable interrupt to any of the 4 TRISC cores in the Tensix. Optionally, the interrupt lines are also routed to the overlay block to pass the information to the SMN. The address and type of error is populated in a set of error registers in the Tensix. The interrupt handler can query the error registers to determine the type of error and optionally issue an ECC scrubbing write to the location that exhibited an error. Double-bit errors also update the error registers.

The L1 banks use an EDC bus to keep track of any ECC errors encountered. The EDC bus is a ring that connects all the L1 SRAM banks and a bus interface unit (BIU). Any ECC errors in the L1 cause an EDC event to be pushed to the EDC bus. When an event reaches the BIU, it can raise mask-able interrupts to the data movement cores. The interrupt handler can read the error details through the BIU and issue a scrubbing write to the address that experienced a single-bit error. The BIU can be read by SMN, NOC or Risc cores (it is mapped to the global arbiter address space). L1 ECC error is also routed to the programable interrupt controllers in the compute cores. These hardware interrupts may be masked by the TRISC cores or handled by an interrupt handler.

NOC header is protected by ECC and the buffers are protected by parity. All tensix RAS events (L1 and TRISC cores) are also routed to the top-level NEO SMN. The EDC buses are highlighted in red in the figure below.

Neo Auto Configuration

NEO Cluster implements multiple features to support functional safety requirements of ASIL-B standard. These hardware features are included after performing failure mode analysis for the NEO Cluster and determining which areas of the design may be susceptible to safety issues. These features are described in the block-level micro-architectural specifications; however a brief summary is shown below.

- Parity protection for register file, including self test ability for each register file
- Self-test ability in packers, unpackers, matrix engine and vector engine

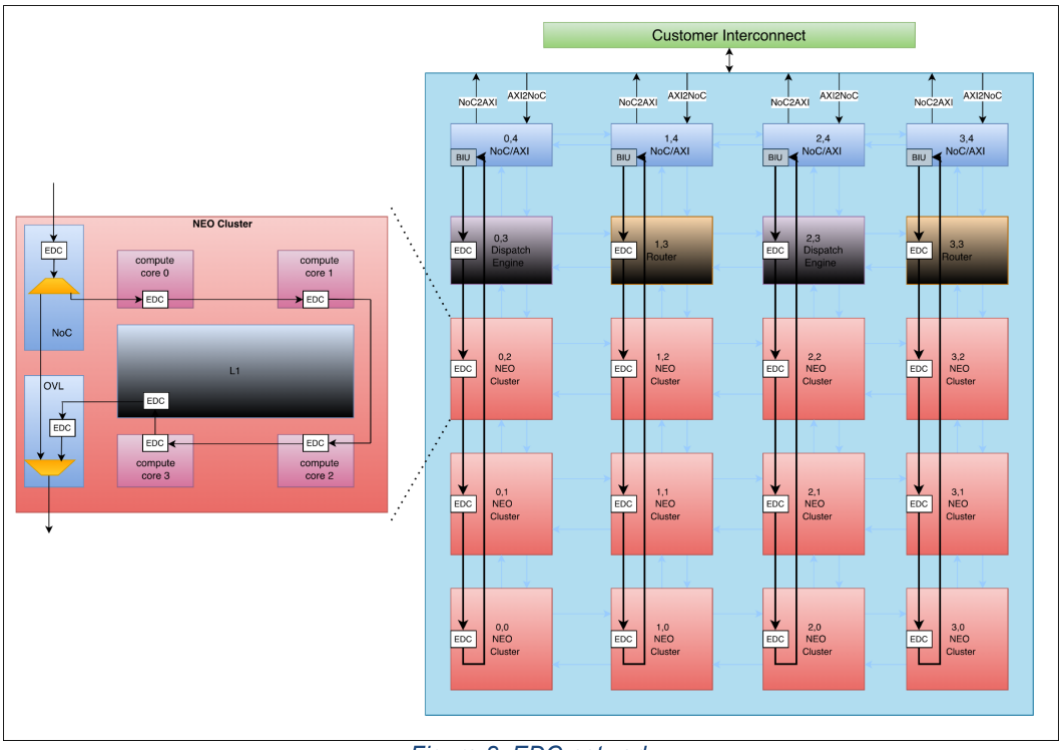

Figure 8: EDC network

- SECDED ECC protection in the L1
- ECC protection in NoC header
- Parity protection of NoC buffers

Functional safety features have a PPA impact. Consequently, these features are configurable in the design, allowing chips designed for non-safety critical applications to exclude them. Functional safety features are managed using a chip-wide EDC network. Figure 8 shows the EDC network connectivity.

Clock Domains

Each NEO Cluster implement three clock domains, AI clock, data-movement clock and NOC clock. For NEO and NEO Robotics, the AI clock is locally generated in each NEO cluster using a digital PLL. The AI clock is used to drive compute cores and the L1. The NOC clock is used for the NIU and the NOC router. The data-movement clock is used to drive the data-movement core cluster. Clock-domain crossing FIFOs are implemented across the interfaces of these domains. For NEO Auto, all clocks are externally generated and delivered to each NEO Cluster.

Power and Thermal Management

Each compute core in NEO implements a performance limiting mechanism that can reduce the maximum instruction issue rate of matrix multiplication instructions. The programmer can specify the peak allowable issue rate using config registers. This is intended to alleviate thermal constraints without reducing AI clock frequency. Reducing AI clock frequency can impact NoC performance since the L1 is within the AI clock domain.

For mitigating di/dt issues, each compute core in NEO can prevent sudden changes in matrix-engine utilization. If a compute core has been idle for a specified number of cycles, it will not be able to issue new matrix multiplication instructions immediately. The hardware issues dummy instructions that do not update the state but gradually increase the activity in the matrix engine. The slope of this gradual activity increase is programable by software. After a period of activity in the matrix engine, if no new instructions are being issued, the hardware issues more dummy instructions to gradually reduce the activity level in the matrix engine.

Neo also adds a droop-prediction mechanism that detects hardware instructions and predicts a droop event when high-power blocks are expected to go active in the next few cycles. The digital PLL is used to dynamically reduce the frequency in such an event. Neo also adds the ability to issue dummy FPU instructions to reduce the di/dt events caused by isolated packer, unpacker or sfpu activity.

Neo utilizes voltage-sensors to constantly monitor voltage at different locations. If the voltage sensors indicate a drop, potentially signaling a di/dt event, the digital PLL may be used to reduce frequency temporarily. Neo therefore has both droop-prediction and droop-detection mechanisms. Additionally, kernels running on data-movement cores or TRISC cores, and NOC can dynamically adjust the AI clock frequency by modifying PLL memory mapped registers. They can also query voltage levels and minimum voltage observed by the sensors to fine-tune power and performance. Di/Dt features are not available in NEO Auto configuration.

Debug Features

Debug Bus

Each compute core in NEO cluster maintains a 256-bit debug bus that daisy chains across all the sub-blocks of the core. Any TRISC core can read the debug bus using the memory-mapped reads. Typical read of the debug bus will proceed as follows. Write to Tensix Debug Bus Read Enable address in the local reg address. Write data should correspond to the debug bus stop and the 4B segment within the stop. Read from Tensix debug registers corresponding to DEBUG bus. Debug bus can also be accessed using Jtag and NoC access.

TRISC Hardware Interrupts

TRISCs can enable multiple hardware interrupts to handle special cases like hangs and unsupported configuration programming. Additionally, it is also possible to enter an interrupt handler every time srcA/SrcB/SrcS data-valid bits toggle. The TRISC core also has access to the register files, and the interrupt handler can dump the contents of register files when new data is unpacked into them. Additionally, ECC errors can also raise interrupts in the TRISC cores, allowing them to issue a ECC scrubbing writes. Tensix Neo ISA also includes a HALT instruction that can be used by the programmer to suspend execution at any point in the program. NoC/SMN reads can be done on the halted compute-core to debug the state of the core. The core can be unhalted by writing an MMIO register using NoC or SMN.

The table below summarizes some key debug features.

Feature	Use-case
Single-step mode	Each compute core can be configured to operate in a single-step execution mode. In this mode, the core will pause execution after each instruction and subsequently enter an interrupt handler. This functionality is invaluable for meticulously examining the state of all register files following the execution of individual instructions, aiding in precise code analysis and debugging.
Data-valid watchpoints	Individual cores can be configured to establish data-valid watchpoints on the data-valid bit of register files. This capability enables the system to record the state of a register file when new data is received by the unpacker or when the destination register file contains final data ready for packing. This feature is particularly useful for identifying the initial point of data corruption in operations that yield unexpected results.

Config watchpoints	Each compute core can be configured to establish watchpoints on configuration registers. Any write operation to a monitored configuration register will trigger an interrupt. This capability is instrumental in debugging race conditions pertaining to configuration register accesses.
Stack overflow protection	Each Trisc core offers the ability to enable stack overflow interrupts. These interrupts are triggered whenever a stack access attempts to exceed a pre-defined limit. Furthermore, the error register will retain the Program Counter (PC) of the instruction that caused the stack overflow. This mechanism is crucial for diagnosing and resolving stack overflow-related corruption bugs, which can be particularly challenging to debug in hardware.
GDB support on triscs	All Trisc cores are equipped with GDB (GNU Debugger) support, enabling users to perform single-step execution and other debugging operations directly on Trisc cores.
Illegal address range access	Trisc cores can be configured with two distinct address ranges designated for legal L1 cache accesses. Any L1 cache access falling outside these specified ranges will trigger an illegal address interrupt. The error register will store the PC of the instruction responsible for the illegal access. Additionally, data movement cores will also support a programmable address range for legal L1 accesses on a per-core basis.
Comprehensive Address Space Access via SMN and NoC	The entire address space of the NEO architecture is accessible via the System Management Network (SMN) and Network-on-Chip (NoC). This comprehensive access allows software to dump and analyze the complete hardware state in the event of a system hang, providing critical insights for root cause analysis. Note that the SMN will not be available in Trinity configuration.
Illegal configuration	Attempts to configure the packer and unpacker with an illegal configuration will trigger an interrupt. The error register will provide a specific error code, which can be used to ascertain the cause of the invalid configuration.
Instruction buffer timeout	Should a Tensix instruction fail to issue within a pre-programmed number of cycles, an instruction buffer timeout interrupt will be raised. The Trisc will then store the PC of the stalled instruction and clear the instruction buffer.
Semaphore protection	Tensix and global semaphores are protected against underflows, overflows, and uninitialized usage. These protections are invaluable for diagnosing the root cause of system hangs in hardware, as they provide clear indications of the kernel and core responsible for semaphore-related errors.

PC Buffer timeout	Each TRISC can access the semaphores, wait for MOP or all instructions of a thread to finish by accessing memory mapped to the pc buffer. Reads to pc buffer are blocking and can cause a hang if a semaphore does not get become available or if a MOP cannot finish. NEO adds a timeout counter to allow outstanding loads to pc buffer to trigger a maskable interrupt to the TRISC core, in case the load latency exceeds the timeout counter.

Table 7: Summary of debug features available on compute-cores